CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Arquitectura en la nube

Arquitectura y Diseño de Software · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 3

Escalabilidad, tolerancia a fallos y seguridad

La nube no hace que tu sistema escale; solo te deja intentarlo

> **¿Qué pasa si alguien apaga ahora mismo la máquina donde corre su aplicación?**

- La instancia es desechable, no tiene nombre
- El disco local se pierde al reiniciar
- La red falla, y hay que diseñar para eso

Lo que vas a poder hacer al terminar la unidad

- Elegir modelo de servicio y de despliegue con criterio
- Diseñar aplicaciones que escalen horizontalmente
- Aplicar los patrones de resiliencia y saber qué resuelve cada uno
- Separar autenticación, autorización y cifrado
- Definir observabilidad que permita diagnosticar
- Arrancar el proyecto TechShop Cloud con criterio

01

1. Qué cambia en la nube

Casi todo lo que suponías dejó de ser cierto

Supuesto del centro de datos propio	Supuesto en la nube
El servidor es estable y tiene nombre	Las instancias son desechables y sin identidad
El disco local persiste	El disco local se pierde al reiniciar
La red interna es confiable	La red falla y hay que diseñar para eso
Escalar es comprar hardware	Escalar es una decisión de diseño y de costo
La capacidad es fija	La capacidad varía, y la factura con ella

El proveedor asegura la nube; tú aseguras lo que pones en ella

Responde el proveedor

- Hardware, virtualización, seguridad física
- Disponibilidad de la región

Respondes tú

- Configuración, identidades, cifrado, tus datos
- Y ahí ocurre casi todo incidente real

Modelos de servicio: dónde termina tu responsabilidad

	IaaS	PaaS	SaaS
Tú administras	SO, runtime, aplicación, datos	Aplicación y datos	Solo la configuración
El proveedor administra	Hardware y virtualización	+ SO y runtime	Todo
Ejemplo	EC2, Compute Engine	App Engine, Elastic Beanstalk	Gmail, Moodle alojado
Control	Máximo	Medio	Mínimo
Costo operativo del equipo	Alto	Medio	Bajo

Modelos de despliegue: cuál elegir y por qué

Modelo	Elígelo cuando	Costo típico
Pública	La elasticidad importa más que el control	Bajo, variable
Privada	Hay requisito legal o de residencia de datos	Alto, fijo
Híbrida	Un núcleo debe quedarse dentro y el resto puede salir	Medio, complejo
Comunitaria	Varias organizaciones comparten regulación	Compartido

Elegir proveedor no es la decisión importante

Necesidad	AWS	Azure	GCP
Cómputo virtual	EC2	Virtual Machines	Compute Engine
Contenedores gestionados	EKS / ECS	AKS	GKE
Identidad y permisos	IAM	Entra ID + RBAC	IAM
Gestión de claves	KMS	Key Vault	Cloud KMS
Mensajería	SQS / SNS	Service Bus	Pub/Sub
Base de datos relacional	RDS	Azure SQL	Cloud SQL

La decisión que sí es arquitectónica: cuánto atarse a servicios propietarios.

02

2. Escalabilidad

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Vertical y horizontal no son dos sabores de lo mismo

Vertical

- Más CPU a la misma máquina
- Tiene techo
- Sigue habiendo UN nodo
- No mejora la tolerancia a fallos

Horizontal

- Más instancias iguales
- Casi sin techo
- Si una cae, quedan las otras
- Exige aplicación sin estado

Sin estado no es una buena práctica: es el requisito que habilita todo

Estado mal ubicado	Dónde debe vivir
Sesión de usuario en memoria	Token firmado (JWT) o caché distribuida
Archivos subidos en disco local	Almacenamiento de objetos (S3, Blob)
Caché local por instancia	Caché compartida
Contador o acumulado en variable	Base de datos o caché
Tareas programadas en un nodo	Programador externo o bloqueo distribuido

Autoescalado: tres definiciones explícitas

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2          # nunca una sola: no habría tolerancia a fallos
  maxReplicas: 10         # techo de gasto
  metrics:
    - type: Resource
      resource: { name: cpu, target: { averageUtilization: 70 } }
```

Escalar la aplicación mueve el cuello de botella, no lo elimina

Escalas la API

De 1 a 4 instancias

Cuatro veces las conexiones

Contra la misma base

Nuevo cuello

Ahora la base es el límite

Respuestas

Réplicas de lectura, caché, pool, CQRS

PREGUNTA

Una API guarda la sesión del usuario en la memoria de la instancia. ¿Qué impide eso?

A El cifrado en reposo de los datos de sesión

B El escalado horizontal: las instancias dejan de ser intercambiables

C La autenticación mediante JWT con refresh token

D El uso de contenedores para el despliegue

03

3. Cuando algo falla

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Tolerar no es lo mismo que recuperarse

Tolerancia a fallos

El sistema sigue operando aunque un componente caiga

Resiliencia

Además se recupera solo, sin intervención humana

La diferencia

Alguien levantándolo a mano de madrugada

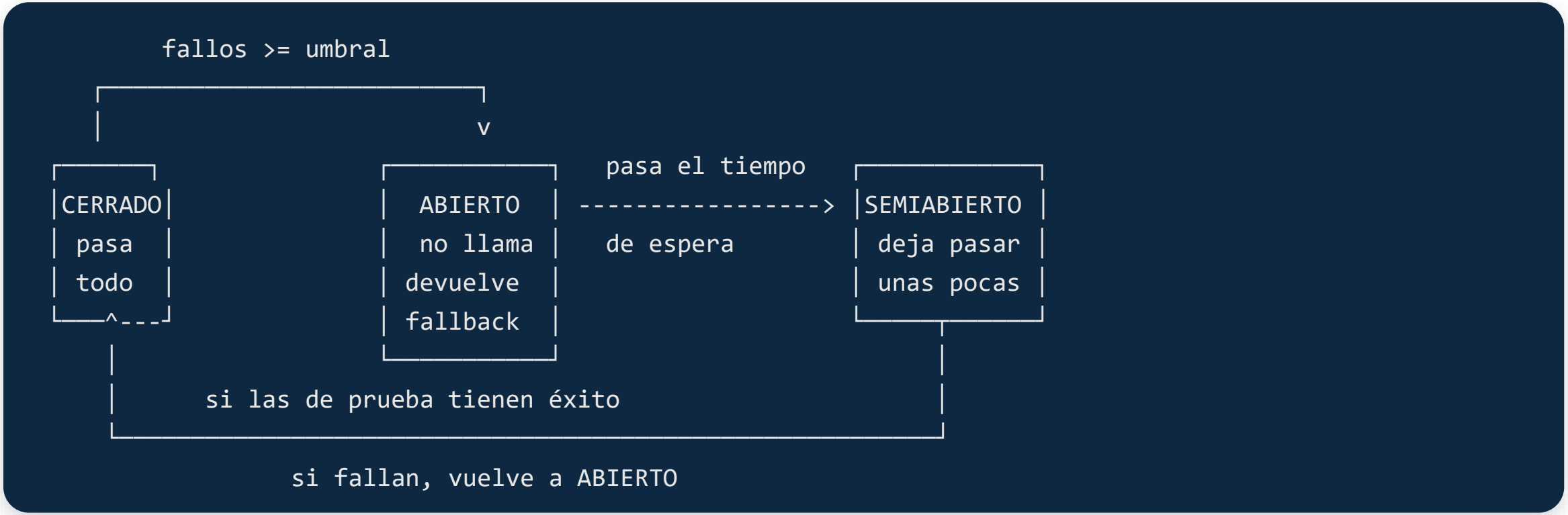
El prerrequisito

Detectar el fallo. Sin observabilidad no hay recuperación

Los siete patrones de resiliencia, y qué resuelve cada uno

Patrón	Qué resuelve	Riesgo si se usa mal
Timeout	Evita esperar indefinidamente	Muy corto: falla lo que iba a funcionar
Reintento con backoff	Supera fallos transitorios	Sin backoff, agrava la saturación
Circuit breaker	Deja de golpear a un servicio caído	Sin *fallback*, solo falla más rápido
Bulkhead	Aísla recursos para que un fallo no consuma todo	Mal dimensionado, desperdicia capacidad
Degradación elegante	Ofrece menos función en vez de ninguna	Requiere decidir qué es prescindible
Health check	Permite sacar de rotación lo enfermo	Si solo responde «vivo», no detecta nada
Idempotencia	Hace seguro el reintento	Sin ella, se duplican cobros

El circuit breaker tiene tres estados



Un circuit breaker sin fallback solo cambia cómo fallas

LA PREGUNTA QUE HAY QUE RESPONDER

```
@CircuitBreaker(name = "pagos", fallbackMethod = "pagoDiferido")
public ResultadoPago cobrar(Pedido p) { return pasarela.cobrar(p); }

private ResultadoPago pagoDiferido(Pedido p, Throwable causa) {
    colaDePagos.encolar(p);                    // degradación elegante
    return ResultadoPago.pendiente("Cobro en proceso");
}
```

REINTENTO CON BACKOFF

```
RetryConfig retry = RetryConfig.custom()
    .maxAttempts(3)
    .intervalFunction(IntervalFunction.ofExponentialBackoff(
        Duration.ofMillis(200), 2.0))       // 200 ms, 400 ms, 800 ms
    .retryExceptions(IOException.class, TimeoutException.class)
    .ignoreExceptions(PagoRechazado.class)  // un rechazo NO se reintenta
    .build();
```

El número de disponibilidad decide la arquitectura

Disponibilidad	Caída máxima al mes	Lo que implica
99 %	7 h 18 min	Un nodo, respaldo diario
99,5 %	3 h 39 min	Redundancia básica, restauración probada
99,9 %	43 min	Multi-zona, conmutación automática
99,99 %	4 min	Multi-región, sin intervención humana

RTO y RPO: dos números que define el negocio

RTO

- Tiempo máximo aceptable sin servicio
- ¿Cuánto podemos estar caídos?
- Decide la estrategia de conmutación

RPO

- Cantidad máxima de datos que se puede perder
- ¿Cuántos minutos de datos?
- Decide la frecuencia de respaldo

PREGUNTA

¿Qué diferencia hay entre tolerancia a fallos y resiliencia?

A Son sinónimos usados en contextos distintos

B La tolerancia aplica a hardware y la resiliencia a software

C Tolerar es seguir operando pese al fallo; ser resiliente es recuperarse sin intervención

D La resiliencia se mide con el RPO y la tolerancia con el RTO

PREGUNTA

Un equipo configura un circuit breaker sobre la pasarela, sin método de fallback. ¿Qué logró?

A Que los pagos se reintenten automáticamente con backoff

B Que la pasarela deje de fallar bajo carga alta

C Que el fallo sea inmediato en lugar de lento, pero siga siendo fallo

D Que el sistema escale cuando la pasarela se degrada

PREGUNTA

El RNF exige 99,9 % de disponibilidad. ¿Qué queda descartado de entrada?

A El uso de contenedores en lugar de máquinas virtuales

B La consistencia eventual entre servicios

C Una única instancia con respaldo y restauración manual

D El autoescalado basado en uso de CPU

04

4. Seguridad

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Autenticar, autorizar y cifrar son tres problemas distintos

Autenticación

¿Quién eres?
Contraseña + MFA, certificados, JWT

Autorización

¿Qué puedes hacer?
RBAC, ABAC, mínimo privilegio

Cifrado

¿Quién puede leerlo?
TLS en tránsito, AES-256 en reposo

El fallo típico

Validar el token y no comprobar el rol

El JWT va codificado, no cifrado

Cabecera

Algoritmo

Carga útil

sub, roles, exp. CUALQUIERA la lee

Firma

Solo garantiza integridad

Sí hace	No hace
Probar que el emisor firmó el token	Ocultar su contenido
Transportar identidad y roles	Poder revocarse antes de expirar
Evitar consultar la sesión en BD	Sustituir a HTTPS

Autorización: el rol no siempre alcanza

Recurso	ADMIN	PROVIDER	USER
Crear producto	Sí	Sí (solo propios)	No
Editar producto	Sí	Sí (solo propios)	No
Ver catálogo	Sí	Sí	Sí
Realizar pedido	Sí	Sí	Sí
Ver pedidos de otros	Sí	No	No
Cambiar roles	Sí	No	No

Las contraseñas no se cifran: se hashean

Cifrar con AES

- Es reversible por definición
- Quien obtenga la clave obtiene TODAS
- Error conceptual

Hashear con bcrypt

- No se descifra, se compara
- Con sal, y lento a propósito
- Encarece la fuerza bruta

Dónde va cada tipo de cifrado, y dónde van los secretos

Dónde	Qué se usa	Detalle
En tránsito	TLS 1.2+	Entre cliente y API, y entre servicios
En reposo	AES-256	Disco, respaldos y campos sensibles
Contraseñas	bcrypt / Argon2	Nunca AES: un hash no se descifra
Claves y secretos	KMS / Key Vault / Secret Manager	Jamás en el código ni en el repositorio

Los cinco riesgos que más aparecen en proyectos como este

1. Control de acceso roto — se valida el token pero no el permiso sobre el recurso

2. Configuración insegura — credenciales por defecto, puertos abiertos, depuración activa

3. Componentes vulnerables — dependencias sin actualizar

4. Fallos de identificación — tokens eternos, sin MFA, sin bloqueo por intentos

5. Registro insuficiente — no queda rastro de quién hizo qué

PREGUNTA

¿Por qué no deben guardarse datos sensibles en la carga útil de un JWT?

A Porque aumentan el tamaño del token y la latencia

B Porque va codificada, no cifrada: cualquiera con el token puede leerla

C Porque impide validar la firma del token en el servidor

D Porque los datos sensibles invalidan el campo de expiración

PREGUNTA

¿Cuál es la forma correcta de almacenar contraseñas de usuarios?

A Cifradas con AES-256 usando una clave en el gestor de secretos

B Con una función de hash lenta y con sal, como bcrypt o Argon2

C Codificadas en Base64 dentro de una columna protegida

D Cifradas con la misma clave privada que firma los JWT

PREGUNTA

En el modelo de responsabilidad compartida, ¿de qué responde el cliente?

A Del hardware, la virtualización y la seguridad física del centro de datos

B De la disponibilidad de la región y de la red del proveedor

C De la configuración, las identidades, el cifrado y sus datos

D De aplicar los parches del hipervisor y del sistema anfitrión

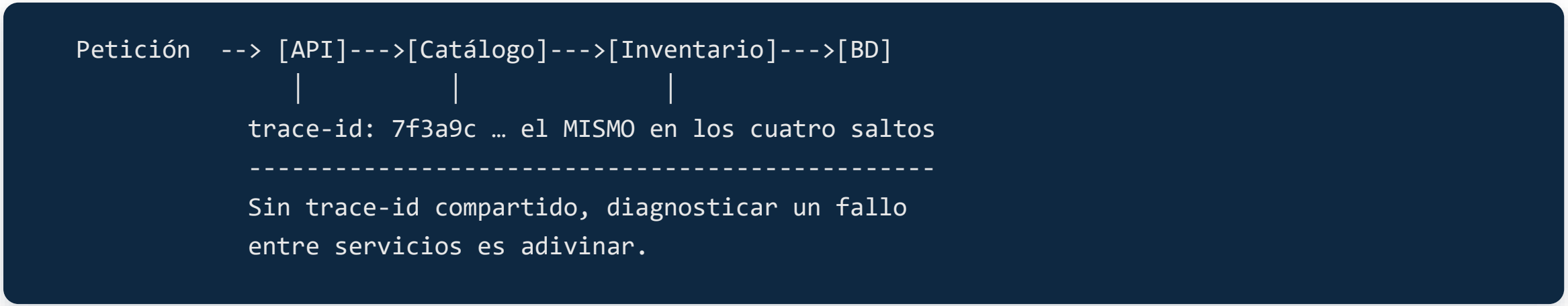
05

5. Observabilidad, cierre y proyecto

Sin observabilidad no hay resiliencia posible

Pilar	Responde	Herramienta típica
Métricas	¿Cómo va el sistema ahora?	Prometheus, CloudWatch
Registros	¿Qué pasó exactamente?	Logs estructurados centralizados
Trazas	¿Por dónde pasó esta petición?	OpenTelemetry, Jaeger

Un identificador de correlación en todos los saltos



PREGUNTA

¿Qué hace que un endpoint de salud sea realmente útil?

A Responder siempre 200 para que el balanceador no saque la instancia

B Devolver la versión de la aplicación y su tiempo de actividad

C Verificar las dependencias críticas, como base de datos y caché

D Exponer las métricas de CPU y memoria de la instancia

Del concepto al proyecto: qué sección cubre qué requisito

Requisito del proyecto	Sección	Peso en la rúbrica
Arquitectura hexagonal + DDD	Unidad 1	Arquitectura 30 %
Casos de uso con responsabilidades separadas	Unidad 1	Funcionalidad 25 %
TDD con cobertura > 80 %	Manual de la unidad	Pruebas 20 %
Docker y Kubernetes	Sección 2 y manual	Arquitectura 30 %
Escalado horizontal automático	Sección 2	Arquitectura 30 %
JWT + RBAC + AES-256	Sección 4	Seguridad 15 %
Circuit breaker y reintento	Sección 3	Seguridad 15 %
Monitoreo y observabilidad	Sección 5	Documentación 10 %

Tres ideas para llevarte

Sin estado o no escalas

Todo lo demás se deriva de ahí

Decide qué pasa al fallar

El fallback es una decisión de negocio

Los números mandan

99,9 % descarta arquitecturas por aritmética

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com